```
import {
  AccessibilityIssue,
  Impact,
  PageScanResult,
  ScanMode,
  ConformanceLevel,
  IssueStatus,
  IssueCategory
} from "../types";

declare const axe: any;

/**
 * Siteimprove Accessibility (SIA) Engine v2.0
 * Comprehensive implementation of all SIA rules with WCAG mappings.
 */
class SIAEngine {
  private config = {
    checkDeprecatedRules: false,
    stopOnFirstError: false,
  };

  private static wcagMapping: Record<
    string,
    { sc: string[]; level: ConformanceLevel }
  > = {
    "SIA-R1": { sc: ["2.4.2"], level: "A" },
    "SIA-R2": { sc: ["1.1.1"], level: "A" },
    "SIA-R3": { sc: ["4.1.1"], level: "A" },
```

"SIA-R4": { sc: ["3.1.1"], level: "A" },

"SIA-R5": { sc: ["3.1.1"], level: "A" },

"SIA-R6": { sc: ["3.1.1"], level: "S" },

"SIA-R7": { sc: ["3.1.2"], level: "AA" },

"SIA-R8": { sc: ["1.3.1", "4.1.2"], level: "A" },

"SIA-R9": { sc: ["2.2.1"], level: "A" },

"SIA-R10": { sc: ["1.3.5"], level: "AA" },

"SIA-R11": { sc: ["2.4.4"], level: "A" },

"SIA-R12": { sc: ["4.1.2"], level: "A" },

"SIA-R13": { sc: ["4.1.2"], level: "A" },

"SIA-R14": { sc: ["2.5.3"], level: "A" },

"SIA-R15": { sc: ["2.4.4"], level: "A" },

"SIA-R16": { sc: ["4.1.2"], level: "A" },

"SIA-R17": { sc: ["4.1.2"], level: "A" },

"SIA-R18": { sc: ["4.1.2"], level: "A" },

"SIA-R19": { sc: ["4.1.2"], level: "A" },

"SIA-R20": { sc: ["4.1.2"], level: "A" },

"SIA-R21": { sc: ["4.1.2"], level: "A" },

"SIA-R22": { sc: ["1.2.2"], level: "A" },

"SIA-R23": { sc: ["1.2.1"], level: "A" },

"SIA-R24": { sc: ["1.2.1"], level: "A" },

"SIA-R25": { sc: ["1.2.3"], level: "A" },

"SIA-R26": { sc: ["1.2.1"], level: "A" },

"SIA-R27": { sc: ["1.2.1"], level: "A" },

"SIA-R28": { sc: ["1.1.1"], level: "A" },

"SIA-R29": { sc: ["1.2.1"], level: "A" },

"SIA-R30": { sc: ["1.2.1"], level: "A" },

"SIA-R31": { sc: ["1.2.1"], level: "A" },

"SIA-R32": { sc: ["1.2.1"], level: "A" },

"SIA-R33": { sc: ["1.2.1"], level: "A" },
"SIA-R34": { sc: ["1.2.3"], level: "S" },
"SIA-R35": { sc: ["1.2.1"], level: "A" },
"SIA-R36": { sc: ["1.2.3"], level: "S" },
"SIA-R37": { sc: ["1.2.3"], level: "A" },
"SIA-R38": { sc: ["1.2.1"], level: "A" },
"SIA-R39": { sc: ["1.1.1"], level: "S" },
"SIA-R40": { sc: ["1.3.1"], level: "A" },
"SIA-R41": { sc: ["2.4.4"], level: "AA" },
"SIA-R42": { sc: ["1.3.1"], level: "A" },
"SIA-R43": { sc: ["1.1.1"], level: "A" },
"SIA-R44": { sc: ["1.3.4"], level: "AA" },
"SIA-R45": { sc: ["1.3.1"], level: "A" },
"SIA-R46": { sc: ["1.3.1"], level: "A" },
"SIA-R47": { sc: ["1.4.4"], level: "AA" },
"SIA-R48": { sc: ["1.4.2"], level: "A" },
"SIA-R49": { sc: ["1.4.2"], level: "A" },
"SIA-R50": { sc: ["1.4.2"], level: "A" },
"SIA-R52": { sc: ["1.1.1"], level: "S" },
"SIA-R53": { sc: ["1.3.1"], level: "A" },
"SIA-R54": { sc: ["4.1.3"], level: "AA" },
"SIA-R55": { sc: ["1.3.1"], level: "S" },
"SIA-R56": { sc: ["1.3.1"], level: "S" },
"SIA-R57": { sc: ["1.3.1"], level: "S" },
"SIA-R58": { sc: ["2.4.1"], level: "A" },
"SIA-R59": { sc: ["2.4.6"], level: "AA" },
"SIA-R60": { sc: ["1.3.1"], level: "A" },
"SIA-R61": { sc: ["2.4.6"], level: "S" },
"SIA-R62": { sc: ["1.4.1"], level: "A" },

"SIA-R63": { sc: ["1.1.1"], level: "A" },
"SIA-R64": { sc: ["2.4.6"], level: "AA" },
"SIA-R65": { sc: ["2.4.7"], level: "AA" },
"SIA-R66": { sc: ["1.4.6"], level: "AAA" },
"SIA-R67": { sc: ["1.1.1"], level: "A" },
"SIA-R68": { sc: ["1.3.1"], level: "A" },
"SIA-R69": { sc: ["1.4.3"], level: "AA" },
"SIA-R70": { sc: ["4.1.1"], level: "S" },
"SIA-R71": { sc: ["1.4.8"], level: "AAA" },
"SIA-R72": { sc: ["1.4.8"], level: "AAA" },
"SIA-R73": { sc: ["1.4.8"], level: "AAA" },
"SIA-R74": { sc: ["1.4.4"], level: "AA" },
"SIA-R75": { sc: ["1.4.4"], level: "AA" },
"SIA-R76": { sc: ["1.3.1"], level: "A" },
"SIA-R77": { sc: ["1.3.1"], level: "A" },
"SIA-R78": { sc: ["1.3.1"], level: "S" },
"SIA-R79": { sc: ["1.3.1"], level: "S" },
"SIA-R80": { sc: ["1.4.4"], level: "AA" },
"SIA-R81": { sc: ["2.4.4"], level: "AA" },
"SIA-R82": { sc: ["3.3.1"], level: "A" },
"SIA-R83": { sc: ["1.4.4"], level: "AA" },
"SIA-R84": { sc: ["2.1.1"], level: "A" },
"SIA-R85": { sc: ["1.4.8"], level: "AAA" },
"SIA-R86": { sc: ["1.1.1"], level: "A" },
"SIA-R87": { sc: ["2.4.1"], level: "A" },
"SIA-R88": { sc: ["1.4.3"], level: "AA" },
"SIA-R89": { sc: ["1.4.6"], level: "AAA" },
"SIA-R90": { sc: ["1.3.1"], level: "A" },
"SIA-R91": { sc: ["1.4.12"], level: "AA" },

```
  "SIA-R92": { sc: ["1.4.12"], level: "AA" },
  "SIA-R93": { sc: ["1.4.12"], level: "AA" },
  "SIA-R94": { sc: ["4.1.2"], level: "A" },
  "SIA-R95": { sc: ["2.1.1"], level: "A" },
  "SIA-R96": { sc: ["2.2.1"], level: "A" },
  "SIA-R97": { sc: ["2.4.1"], level: "S" },
  "SIA-R98": { sc: ["2.4.6"], level: "S" },
  "SIA-R99": { sc: ["1.3.1"], level: "A" },
  "SIA-R100": { sc: ["2.4.1"], level: "A" },
  "SIA-R101": { sc: ["2.4.1"], level: "S" },
  "SIA-R102": { sc: ["2.4.1"], level: "A" },
  "SIA-R103": { sc: ["1.4.3"], level: "AA" },
  "SIA-R104": { sc: ["1.4.6"], level: "AAA" },
  "SIA-R109": { sc: ["3.1.1"], level: "A" },
  "SIA-R110": { sc: ["4.1.2"], level: "A" },
  "SIA-R111": { sc: ["2.5.5"], level: "AAA" },
  "SIA-R113": { sc: ["2.5.5"], level: "AAA" },
  "SIA-R114": { sc: ["2.4.2"], level: "A" },
  "SIA-R115": { sc: ["2.4.6"], level: "AA" },
  "SIA-R116": { sc: ["4.1.2"], level: "A" },
  "SIA-R117": { sc: ["1.1.1"], level: "A" },
};
private isVisible(el: Element, win: Window): boolean {
  if (!(el instanceof HTMLElement)) return false;

  // Hidden via attribute
  if (el.hasAttribute("hidden")) return false;

  // aria-hidden
```

```
  if (el.closest('[aria-hidden="true"]')) return false;

  // Computed styles
  let current: HTMLElement | null = el;
  while (current) {
    const style = win.getComputedStyle(current);

    if (
      style.display === "none" ||
      style.visibility === "hidden" ||
      style.opacity === "0"
    )
      return false;

    current = current.parentElement;
  }

  // Size check
  const rect = el.getBoundingClientRect();
  if (rect.width === 0 || rect.height === 0) return false;

  // Offscreen large negative positioning
  if (rect.bottom < 0 || rect.right < 0) return false;

  return true;
}

private getAccessibleName(element: Element, doc: Document): string {
  if (!(element instanceof HTMLElement)) return "";
```

```
// aria-labelledby
const labelledBy = element.getAttribute("aria-labelledby");
if (labelledBy) {
  return labelledBy
    .split(/\s+/)
    .map((id) => doc.getElementById(id)?.textContent?.trim() || "")
    .join(" ")
    .trim();
}

// aria-label
const ariaLabel = element.getAttribute("aria-label");
if (ariaLabel) return ariaLabel.trim();

// Native label association (for inputs)
if (
  element instanceof HTMLInputElement ||
  element instanceof HTMLSelectElement ||
  element instanceof HTMLTextAreaElement
) {
  const id = element.id;
  if (id) {
    const label = doc.querySelector(`label[for="${id}"]`);
    if (label) return label.textContent?.trim() || "";
  }

  if (element.closest("label")) {
    return element.closest("label")?.textContent?.trim() || "";
```

```
    }
  }

  // Alt attribute (images)
  if (element instanceof HTMLImageElement) {
    return element.alt?.trim() || "";
  }

  // Title attribute fallback
  const title = element.getAttribute("title");
  if (title) return title.trim();

  // Text content (excluding hidden)
  return this.getVisibleText(element).trim();
}

private getVisibleText(element: Element): string {
  let text = "";

  const walker = document.createTreeWalker(
    element,
    NodeFilter.SHOW_TEXT,
    null,
  );

  let node;
  while ((node = walker.nextNode())) {
    const parent = node.parentElement;
    if (!parent) continue;
```

```
    const style = window.getComputedStyle(parent);
    if (
      style.display === "none" ||
      style.visibility === "hidden" ||
      style.opacity === "0"
    )
      continue;

    if (parent.closest('[aria-hidden="true"]')) continue;

    text += node.textContent;
  }
  return text;
}

private getLuminance(color: string): number | null {
  const rgb = this.parseColor(color);
  if (!rgb) return null;
  const [r, g, b] = rgb.map((c) => {
    c = c / 255;
    return c <= 0.03928 ? c / 12.92 : Math.pow((c + 0.055) / 1.055, 2.4);
  });
  return 0.2126 * r + 0.7152 * g + 0.0722 * b;
}

private parseColor(color: string): number[] | null {
  if (!color) return null;
```

```
  // rgb / rgba
  const rgbMatch = color.match(/rgba?\((\d+),\s*(\d+),\s*(\d+)/);
  if (rgbMatch) {
    return [
      parseInt(rgbMatch[1]),
      parseInt(rgbMatch[2]),
      parseInt(rgbMatch[3]),
    ];
  }

  // hex format
  if (color.startsWith("#")) {
    let hex = color.replace("#", "");

    if (hex.length === 3) {
      hex = hex
        .split("")
        .map((c) => c + c)
        .join("");
    }

    const bigint = parseInt(hex, 16);
    return [(bigint >> 16) & 255, (bigint >> 8) & 255, bigint & 255];
  }

  return null;
}

private getContrastRatio(color1: string, color2: string): number | null {
```

```
  const luminance1 = this.getLuminance(color1);
  const luminance2 = this.getLuminance(color2);

  if (luminance1 === null || luminance2 === null) return null;

  const lighter = Math.max(luminance1, luminance2);
  const darker = Math.min(luminance1, luminance2);

  return (lighter + 0.05) / (darker + 0.05);
}

private checkContrast(doc: Document, win: Window, enhanced: boolean = false) {
  const textElements = doc.querySelectorAll("body *");
  const failures: Element[] = [];

  textElements.forEach((element) => {
    if (!(element instanceof HTMLElement)) return;

    // Visibility filter (ADD THIS)
    if (!this.isVisible(element, win)) return;

    // Must contain real text
    if (!element.textContent?.trim()) return;

    const style = win.getComputedStyle(element);

    const color = style.color;
    let bgColor = style.backgroundColor;
    let parent = element.parentElement;
```

```
  while (
    (bgColor === "transparent" || bgColor === "rgba(0, 0, 0, 0)") &&
    parent
  ) {
    const parentStyle = win.getComputedStyle(parent);
    bgColor = parentStyle.backgroundColor;
    parent = parent.parentElement;
  }

  if (bgColor === "transparent" || bgColor === "rgba(0, 0, 0, 0)") {
    bgColor = "rgb(255,255,255)"; // default to white only if truly no bg
  }

  const fontSize = parseFloat(style.fontSize);
  const fontWeight = parseFloat(style.fontWeight);
  const isLarge = fontSize >= 18 || (fontSize >= 14 && fontWeight >= 700);
  const minRatio = enhanced ? (isLarge ? 4.5 : 7) : isLarge ? 3 : 4.5;
  const ratio = this.getContrastRatio(color, bgColor);
  if (ratio === null) return;
  if (ratio < minRatio) {
    failures.push(element);
  }
});

return {
  passed: failures.length === 0,
  elements: failures,
  message:
```

```
      failures.length === 0
        ? `All text has ${enhanced ? "enhanced" : "minimum"} contrast`
        : `${failures.length} element(s) have text without ${enhanced ? "enhanced" : "minimum"} contrast`,
    recommendation:
      failures.length > 0
        ? `Improve color contrast to meet ${enhanced ? "enhanced" : "minimum"} requirements`
        : null,
  };
}
private isInlineTextLink(el: HTMLElement): boolean {
  if (el.tagName.toLowerCase() !== "a") return false;

  // If link is inside paragraph and has no block display
  const parent = el.parentElement;
  if (!parent) return false;

  const style = window.getComputedStyle(el);

  const isInline =
    style.display === "inline" || style.display === "inline-block";

  const insideTextContainer =
    parent.tagName.toLowerCase() === "p" ||
    parent.tagName.toLowerCase() === "span";

  return isInline && insideTextContainer;
}
private initializeRules(win: Window): Record<string, any> {
  const doc = win.document;
```

```
const rules: Record<string, any> = {};


rules["SIA-R1"] = {
  title: "Documents have a <title> element",
  severity: "critical",
  check: (d: Document) => {
    const t = d.querySelector("head > title");
    const has = !!(t && t.textContent?.trim());
    return {
      passed: has,
      elements: has ? [] : [d.documentElement],
      message: has ? "Document has a title" : "Missing title element",
    };
  },
};
rules["SIA-R2"] = {
  title: "Images have an accessible name",
  severity: "critical",
  check: (d: Document) => {
    const imgs = d.querySelectorAll(
      'img:not([aria-hidden="true"]):not([role="presentation"]):not([role="none"])',
    );
    const fails = Array.from(imgs).filter(
      (img) =>
        !(
          img.getAttribute("alt") !== null ||
          img.getAttribute("aria-label") ||
          img.getAttribute("aria-labelledby") ||
          img.getAttribute("title")
```

```
      ),
    );
    return {
      passed: fails.length === 0,
      elements: fails,
      message:
        fails.length === 0
          ? "All images have names"
          : `${fails.length} images missing names`,
    };
  },
};
rules["SIA-R3"] = {
  title: "id attributes have a unique value",
  severity: "critical",
  check: (d: Document) => {
    const ids = d.querySelectorAll("[id]");
    const map = new Map();
    const dups: Element[] = [];
    ids.forEach((el) => {
      const id = el.getAttribute("id");
      if (id) {
        if (map.has(id)) {
          dups.push(map.get(id));
          dups.push(el);
        } else map.set(id, el);
      }
    });
    return {
```

```
      passed: dups.length === 0,
      elements: dups,
      message:
        dups.length === 0
          ? "All IDs unique"
          : "Duplicate ID attributes found",
    };
  },
};
rules["SIA-R4"] = {
  title: "<html> document elements have a lang attribute",
  severity: "critical",
  check: (d: Document) => {
    const h = d.documentElement;
    const has = h.hasAttribute("lang");
    return {
      passed: has,
      elements: has ? [] : [h],
      message: has ? "Has lang" : "Missing lang",
    };
  },
};
rules["SIA-R5"] = {
  title: "<html> document elements have a valid lang attribute",
  severity: "critical",
  check: (d: Document) => {
    const h = d.documentElement;
    const l = h.getAttribute("lang");
    const valid = /^[a-z]{2,3}(-[A-Z]{2})?$/.test(l || "");
```

```
    return {
      passed: valid,
      elements: valid ? [] : [h],
      message: valid ? "Valid lang" : "Invalid lang format",
    };
  },
};
rules["SIA-R6"] = {
  title:
    "<html> document elements have matching lang and xml:lang attributes (DEPRECATED)",
  severity: "serious",
  check: (d: Document) => {
    const h = d.documentElement;
    const l = h.getAttribute("lang");
    const xl = h.getAttribute("xml:lang");
    const p = !xl || l === xl;
    return {
      elements: !xl || l === xl ? [] : [h],
      message: p ? "Matching langs" : "Langs do not match",
    };
  },
};
rules["SIA-R7"] = {
  title: "lang attributes within the <body> element have a valid value",
  severity: "serious",
  check: (d: Document) => {
    const els = d.querySelectorAll("body [lang]");
    const fails: Element[] = [];
    els.forEach((el) => {
```

```
      if (!/^[a-z]{2,3}(-[A-Z]{2})?$/.test(el.getAttribute("lang") || ""))
        fails.push(el);
    });
    return { elements: fails, message: "Valid body langs" };
  },
};
rules["SIA-R8"] = {
  title: "Form fields have an accessible name",
  severity: "critical",
  check: (d: Document) => {
    const fields = d.querySelectorAll(
      'input:not([type="hidden"]), textarea, select, [role="textbox"], [role="combobox"], [role="listbox"], [role="slider"], [role="spinbutton"]',
    );
    const fails = Array.from(fields).filter(
      (f) =>
        !(
          (f as any).labels?.length > 0 ||
          f.hasAttribute("aria-label") ||
          f.hasAttribute("aria-labelledby") ||
          f.hasAttribute("title")
        ),
    );
    return {
      passed: fails.length === 0,
      elements: fails,
      message:
        fails.length === 0
          ? "All fields have names"
```

```
        : "Form fields missing names",
    };
  },
};
rules["SIA-R9"] = {
  title: "Refreshes implemented using the <meta> element have no delay",
  severity: "critical",
  check: (d: Document) => {
    const m = d.querySelector('meta[http-equiv="refresh"]');
    if (!m) return { elements: [], message: "No meta refresh" };
    const c = m.getAttribute("content") || "";
    const delay = parseInt(c.match(/(\d+)/)?.[1] || "0");
    return {
      passed: delay === 0,
      elements: delay > 0 ? [m] : [],
      message: delay > 0 ? "Meta refresh delay found" : "No delay",
    };
  },
};
rules["SIA-R10"] = {
  title: "autocomplete attributes have a valid value",
  severity: "serious",
  check: (d: Document) => {
    const fields = d.querySelectorAll("[autocomplete]");
    const valid = ["name", "email", "username", "on", "off"];
    const fails = Array.from(fields).filter(
      (f) => !valid.includes(f.getAttribute("autocomplete") || ""),
    );
    return { elements: fails, message: "Autocomplete check" };
```

```
  },
};
rules["SIA-R11"] = {
  title: "Links have an accessible name",
  severity: "critical",
  check: (d: Document) => {
    const links = d.querySelectorAll('a:not([aria-hidden="true"])');
    const fails = Array.from(links).filter(
      (l) => this.isVisible(l, win) && !this.getAccessibleName(l, doc),
    );
    return {
      passed: fails.length === 0,
      elements: fails,
      message: "Links missing names",
    };
  },
};
rules["SIA-R12"] = {
  title: "Buttons have an accessible name",
  severity: "critical",
  check: (d: Document) => {
    const btns = d.querySelectorAll(
      'button:not([aria-hidden="true"]), input[type="button"], [role="button"]',
    );
    const fails = Array.from(btns).filter(
      (b) => this.isVisible(l, win) && !this.getAccessibleName(b, doc),
    );
    return {
      passed: fails.length === 0,
```

```
      elements: fails,
      message: "Buttons missing names",
    };
  },
};
rules["SIA-R13"] = {
  title: "<iframe> elements have an accessible name",
  severity: "critical",
  check: (d: Document) => {
    const frames = d.querySelectorAll('iframe:not([aria-hidden="true"])');
    const fails = Array.from(frames).filter(
      (f) => !this.getAccessibleName(f, doc),
    );
    return {
      passed: fails.length === 0,
      elements: fails,
      message: "Iframes missing names",
    };
  },
};
rules["SIA-R14"] = {
  title: "Visible labels are included in accessible names",
  severity: "critical",
  check: (d: Document) => {
    const fields = d.querySelectorAll(
      'input:not([type="hidden"]), textarea, select',
    );
    const fails = Array.from(fields).filter((f) => {
      const label = (f as any).labels?.[0]?.textContent?.trim();
```

```
      return label && !this.getAccessibleName(f, doc).includes(label);
    });
    return {
      passed: fails.length === 0,
      elements: fails,
      message: "Label mismatch",
    };
  },
};
rules["SIA-R15"] = {
  title: "<iframe> elements with identical names purpose",
  severity: "serious",
  check: (d: Document) => {
    const frames = d.querySelectorAll("iframe[title]");
    const map = new Map();
    const warns: Element[] = [];
    frames.forEach((f) => {
      const t = f.getAttribute("title");
      if (t && map.has(t) && map.get(t).src !== (f as any).src)
        warns.push(f);
      else map.set(t, f);
    });
    return { elements: warns, message: "Iframe names purpose check" };
  },
};
rules["SIA-R16"] = {
  title: "Elements with a role have required states and properties",
  severity: "critical",
  check: (d: Document) => {
```

```
    const roles = d.querySelectorAll("[role]");
    const requirements: any = {
      checkbox: ["aria-checked"],
      combobox: ["aria-expanded"],
      slider: ["aria-valuenow"],
    };
    const fails: Element[] = [];
    roles.forEach((el) => {
      const r = el.getAttribute("role") || "";
      if (
        requirements[r] &&
        !requirements[r].every((a: string) => el.hasAttribute(a))
      )
        fails.push(el);
    });
    return {
      passed: fails.length === 0,
      elements: fails,
      message: "Missing required ARIA",
    };
  },
};
rules["SIA-R17"] = {
  title: 'Elements with aria-hidden="true" are not focusable',
  severity: "critical",
  check: (d: Document) => {
    const hid = d.querySelectorAll('[aria-hidden="true"]');
    const fails = Array.from(hid).filter(
      (el) =>
```

```
      el.hasAttribute("tabindex") && el.getAttribute("tabindex") !== "-1",
    );
    return {
      passed: fails.length === 0,
      elements: fails,
      message: "Hidden element focusable",
    };
  },
};
rules["SIA-R18"] = {
  title: "aria-* states and properties are allowed",
  severity: "critical",
  check: (d: Document) => {
    const els = d.querySelectorAll("*");
    const fails: Element[] = [];
    els.forEach((el) => {
      Array.from(el.attributes).forEach((a) => {
        if (
          a.name.startsWith("aria-") &&
          ["aria-label", "aria-labelledby", "aria-hidden"].includes(a.name)
        )
          fails.push(el);
      });
    });
    return { elements: fails, message: "Uncommon ARIA used" };
  },
};
rules["SIA-R19"] = {
  title: "aria-* states and properties have a valid value",
```

```
  severity: "critical",
  check: (d: Document) => {
    const els = d.querySelectorAll("[aria-checked], [aria-expanded]");
    const fails: Element[] = [];
    els.forEach((el) => {
      ["aria-checked", "aria-expanded"].forEach((a) => {
        if (
          el.hasAttribute(a) &&
          ![“true”, “false”, “mixed”].includes(el.getAttribute(a) || “”)
        )
          fails.push(el);
      });
    });
    return {
      passed: fails.length === 0,
      elements: fails,
      message: "Invalid ARIA value",
    };
  },
};
rules["SIA-R20"] = {
  title: "aria-* attributes have a valid name",
  severity: "critical",
  check: (d: Document) => {
    return { elements: [], message: "ARIA name check" };
  },
};
rules["SIA-R21"] = {
  title: "role attributes have only valid values",
```

```
severity: "critical",
check: (d: Document) => {
  const roles = d.querySelectorAll("[role]");
  const valid = [
    "alert",
    "button",
    "checkbox",
    "dialog",
    "form",
    "grid",
    "heading",
    "img",
    "link",
    "list",
    "main",
    "navigation",
    "region",
    "search",
    "table",
    "textbox",
    "menu",
    "menuitem",
    "tab",
    "tablist",
    "tabpanel",
  ];
  const fails = Array.from(roles).filter((el) =>
    el
      .getAttribute("role")
```

```
      ?.split(" ")
      .some((r) => !valid.includes(r)),
    );
    return {
      passed: fails.length === 0,
      elements: fails,
      message: "Invalid role",
    };
  },
};
rules["SIA-R22"] = {
  title: "<video> auditory has captions",
  severity: "critical",
  check: (d: Document) => {
    const vids = d.querySelectorAll("video");
    const fails = Array.from(vids).filter(
      (v) => !v.querySelector('track[kind="captions"]'),
    );
    return {
      passed: fails.length === 0,
      elements: fails,
      message: "Video missing captions",
    };
  },
};
rules["SIA-R23"] = {
  title: "<audio> has transcript",
  severity: "serious",
  check: (d: Document) => {
```

```
    return {
      elements: Array.from(d.querySelectorAll("audio")),
      message: "Manual transcript check",
    };
  },
};
rules["SIA-R24"] = {
  title: "<video> visual has transcript",
  severity: "serious",
  check: (d: Document) => {
    return {
      elements: Array.from(d.querySelectorAll("video")),
      message: "Manual transcript check",
    };
  },
};
rules["SIA-R25"] = {
  title: "<video> visual has audio description",
  severity: "serious",
  check: (d: Document) => {
    return {
      elements: Array.from(d.querySelectorAll("video")),
      message: "Manual audio desc check",
    };
  },
};
rules["SIA-R26"] = {
  title: "<video> visual-only content alternative",
  severity: "serious",
```

```
  check: (d: Document) => {
    return { elements: [], message: "Manual media alt check" };
  },
};
rules["SIA-R27"] = {
  title: "<video> auditory accessible alternative",
  severity: "critical",
  check: (d: Document) => {
    return { elements: [], message: "Manual auditory alt check" };
  },
};
rules["SIA-R28"] = {
  title: '<input type="image"> names',
  severity: "critical",
  check: (d: Document) => {
    const inputs = d.querySelectorAll('input[type="image"]');
    const fails = Array.from(inputs).filter(
      (i) => !this.getAccessibleName(i, doc),
    );
    return {
      passed: fails.length === 0,
      elements: fails,
      message: "Image inputs missing names",
    };
  },
};
rules["SIA-R29"] = {
  title: "<audio> content media alternative",
  severity: "serious",
```

```
  check: (d: Document) => {
    return { elements: [], message: "Manual audio media alt check" };
  },
};
rules["SIA-R30"] = {
  title: "<audio> content text alternative",
  severity: "critical",
  check: (d: Document) => {
    return { elements: [], message: "Manual text alt check" };
  },
};
rules["SIA-R31"] = {
  title: "<video> content media alternative",
  severity: "serious",
  check: (d: Document) => {
    return { elements: [], message: "Manual video media alt check" };
  },
};
rules["SIA-R32"] = {
  title: "<video> visual-only audio track",
  severity: "serious",
  check: (d: Document) => {
    return { elements: [], message: "Manual visual-only check" };
  },
};
rules["SIA-R33"] = {
  title: "<video> visual-only transcript",
  severity: "serious",
  check: (d: Document) => {
```

```
    return { elements: [], message: "Manual visual-only transcript" };
  },
};
rules["SIA-R34"] = {
  title: "<video> visual-only description (DEPRECATED)",
  severity: "serious",
  check: (d: Document) => {
    return { elements: [], message: "Deprecated" };
  },
};
rules["SIA-R35"] = {
  title: "<video> visual-only alternative",
  severity: "critical",
  check: (d: Document) => {
    return { elements: [], message: "Manual alt check" };
  },
};
rules["SIA-R36"] = {
  title: "<video> visual description (DEPRECATED)",
  severity: "serious",
  check: (d: Document) => {
    return { elements: [], message: "Deprecated" };
  },
};
rules["SIA-R37"] = {
  title: "<video> strict alternative",
  severity: "critical",
  check: (d: Document) => {
    return { elements: [], message: "Manual strict check" };
```

```
  },
};
rules["SIA-R38"] = {
  title: "<video> visual alternative",
  severity: "critical",
  check: (d: Document) => {
    return { elements: [], message: "Manual alternative check" };
  },
};
rules["SIA-R39"] = {
  title: "Image filename is name",
  severity: "serious",
  check: (d: Document) => {
    const imgs = d.querySelectorAll("img[src][alt]");
    const warns: Element[] = [];
    imgs.forEach((img) => {
      const a = img.getAttribute("alt") || "";
      if (a && img.getAttribute("src")?.includes(a)) warns.push(img);
    });
    return { elements: warns, message: "Alt text matches filename" };
  },
};
rules["SIA-R40"] = {
  title: "Regions have an accessible name",
  severity: "critical",
  check: (d: Document) => {
    const regs = d.querySelectorAll('[role="region"], section[aria-label]');
    const fails = Array.from(regs).filter(
      (r) => !this.getAccessibleName(r, doc),
```

```
    );
    return {
      passed: fails.length === 0,
      elements: fails,
      message: "Regions missing names",
    };
  },
};
rules["SIA-R41"] = {
  title: "Links identical names purpose",
  severity: "serious",
  check: (d: Document) => {
    return { elements: [], message: "Manual link purpose" };
  },
};
rules["SIA-R42"] = {
  title: "Elements with a role required parent",
  severity: "critical",
  check: (d: Document) => {
    const items = d.querySelectorAll('[role="listitem"]');
    const fails = Array.from(items).filter(
      (el) => !el.closest('[role="list"]'),
    );
    return {
      passed: fails.length === 0,
      elements: fails,
      message: "Orphaned listitem",
    };
  },
```

```
};
rules["SIA-R43"] = {
  title: "<svg> with role has name",
  severity: "critical",
  check: (d: Document) => {
    const svgs = d.querySelectorAll("svg[role]");
    const fails = Array.from(svgs).filter(
      (s) => !this.getAccessibleName(s, doc),
    );
    return {
      passed: fails.length === 0,
      elements: fails,
      message: "SVG with role missing name",
    };
  },
};
rules["SIA-R44"] = {
  title: "Orientation restricted",
  severity: "serious",
  check: (d: Document) => {
    return { elements: [], message: "Orientation check" };
  },
};
rules["SIA-R45"] = {
  title: "headers attribute cell table",
  severity: "critical",
  check: (d: Document) => {
    return { elements: [], message: "Headers cell check" };
  },
```

```
};
rules["SIA-R46"] = {
  title: "Header cells data cells",
  severity: "critical",
  check: (d: Document) => {
    return { elements: [], message: "Header cells data" };
  },
};
rules["SIA-R47"] = {
  title: '<meta name="viewport"> zoom',
  severity: "critical",
  check: (d: Document) => {
    const m = d.querySelector('meta[name="viewport"]');
    const c = m?.getAttribute("content") || "";
    const p =
      c.includes("user-scalable=no") || c.includes("maximum-scale=1");
    return {
      passed: !p,
      elements: p ? [m!] : [],
      message: p ? "Prevents zoom" : "Allows zoom",
    };
  },
};
rules["SIA-R48"] = {
  title: "Autoplay audio length",
  severity: "critical",
  check: (d: Document) => {
    return { elements: [], message: "Manual autoplay check" };
  },
```

```
};
rules["SIA-R49"] = {
  title: "Autoplay control",
  severity: "critical",
  check: (d: Document) => {
    return { elements: [], message: "Manual control check" };
  },
};
rules["SIA-R50"] = {
  title: "Avoid automatically playing audio",
  severity: "serious",
  check: (d: Document) => {
    return {
      elements: Array.from(d.querySelectorAll("[autoplay]")),
      message: "Manual avoid autoplay",
    };
  },
};
rules["SIA-R52"] = {
  title: "Adjacent links same resource",
  severity: "serious",
  check: (d: Document) => {
    return { elements: [], message: "Adjacent links" };
  },
};
rules["SIA-R53"] = {
  title: "Headings structured",
  severity: "serious",
  check: (d: Document) => {
```

```
    const hs = Array.from(d.querySelectorAll("h1, h2, h3, h4, h5, h6"));
    const warns: Element[] = [];
    let prev = 0;
    hs.forEach((h) => {
      const l = parseInt(h.tagName[1]);
      if (l > prev + 1) warns.push(h);
      prev = l;
    });
    return { elements: warns, message: "Heading skips" };
  },
};
rules["SIA-R54"] = {
  title: "Assertive marked atomic",
  severity: "serious",
  check: (d: Document) => {
    return { elements: [], message: "Live atomic" };
  },
};
rules["SIA-R55"] = {
  title: "Landmark identical names purpose",
  severity: "serious",
  check: (d: Document) => {
    return { elements: [], message: "Landmark purpose" };
  },
};
rules["SIA-R56"] = {
  title: "Landmarks unique name",
  severity: "serious",
  check: (d: Document) => {
```

```
    return { elements: [], message: "Landmark unique" };
  },
};
rules["SIA-R57"] = {
  title: "Perceivable text included in landmark",
  severity: "serious",
  check: (d: Document) => {
    const walker = d.createTreeWalker(d.body, NodeFilter.SHOW_TEXT, null);
    const failures: Element[] = [];
    const landmarks = [
      "main",
      "nav",
      "header",
      "footer",
      "aside",
      '[role="main"]',
      '[role="navigation"]',
      '[role="banner"]',
      '[role="contentinfo"]',
      '[role="region"]',
    ].join(",");

    let node;
    while ((node = walker.nextNode())) {
      const parent = node.parentElement;
      if (!parent) continue;

      const text = node.textContent?.trim();
      if (!text) continue;
```

```
      // Skip script/style
      if (parent.closest("script, style, template, noscript")) continue;

      // Visibility filter
      if (!this.isVisible(parent, win)) continue;

      // Check landmark containment
      if (!parent.closest(landmarks)) {
        failures.push(parent);
      }
    }

    return {
      elements: failures,
      message:
        failures.length > 0
          ? `${failures.length} text node(s) not inside a landmark`
          : "All perceivable text is inside landmarks",
    };
  },
};
rules["SIA-R58"] = {
  title: "Repeated blocks bypassed",
  severity: "critical",
  check: (d: Document) => {
    const s = d.querySelectorAll('a[href^="#"]');
    return {
      passed: s.length > 0,
```

```
      elements: s.length === 0 ? [d.body] : [],
      message: "Skip link presence",
    };
  },
};
rules["SIA-R59"] = {
  title: "Documents have headings",
  severity: "serious",
  check: (d: Document) => {
    const hs = d.querySelectorAll("h1, h2, h3, h4, h5, h6");
    return {
      passed: hs.length > 0,
      elements: hs.length === 0 ? [d.body] : [],
      message: "Heading presence",
    };
  },
};
rules["SIA-R60"] = {
  title: "Groups have name",
  severity: "critical",
  check: (d: Document) => {
    const gs = d.querySelectorAll('[role="group"]');
    const fails = Array.from(gs).filter(
      (g) => !this.getAccessibleName(g, doc),
    );
    return {
      passed: fails.length === 0,
      elements: fails,
      message: "Groups missing names",
```

```
    };
  },
};
rules["SIA-R61"] = {
  title: "Documents start with H1",
  severity: "serious",
  check: (d: Document) => {
    const h1 = d.querySelector("h1");
    return {
      passed: !!h1,
      elements: !h1 ? [d.body] : [],
      message: "H1 presence",
    };
  },
};
rules["SIA-R62"] = {
  title: "Links distinguishable",
  severity: "serious",
  check: (d: Document) => {
    return { elements: [], message: "Link distinguished" };
  },
};
rules["SIA-R63"] = {
  title: "<object> have name",
  severity: "critical",
  check: (d: Document) => {
    const obs = d.querySelectorAll("object");
    const fails = Array.from(obs).filter(
      (o) => !this.getAccessibleName(o, doc),
```

```
    );
    return {
      passed: fails.length === 0,
      elements: fails,
      message: "Objects missing names",
    };
  },
};
rules["SIA-R64"] = {
  title: "Heading has non-empty name",
  severity: "critical",
  check: (d: Document) => {
    const hs = d.querySelectorAll("h1, h2, h3, h4, h5, h6");
    const fails = Array.from(hs).filter(
      (h) => !this.getAccessibleName(h, doc),
    );
    return {
      passed: fails.length === 0,
      elements: fails,
      message: "Empty headings",
    };
  },
};
rules["SIA-R65"] = {
  title: "Element in sequential focus order focusable",
  severity: "critical",
  check: (d: Document) => {
    return { elements: [], message: "Focus order" };
  },
```

```
};
rules["SIA-R66"] = {
  title: "Text enhanced contrast",
  severity: "serious",
  check: (d: Document) => this.checkContrast(d, win, true),
};
rules["SIA-R67"] = {
  title: "Decorative images not exposed",
  severity: "critical",
  check: (d: Document) => {
    const imgs = d.querySelectorAll('img[alt=""]');
    const fails = Array.from(imgs).filter(
      (el) =>
        (el as Element).hasAttribute("aria-label") ||
        (el as Element).hasAttribute("title"),
    );
    return {
      passed: fails.length === 0,
      elements: fails,
      message: "Decorative but exposed",
    };
  },
};
rules["SIA-R68"] = {
  title: "Elements with a role have required children",
  severity: "critical",
  check: (d: Document) => {
    const list = d.querySelectorAll('[role="list"]');
    const fails = Array.from(list).filter(
```

```
      (l) => !l.querySelector('[role="listitem"]'),
    );
    return {
      passed: fails.length === 0,
      elements: fails,
      message: "Missing required children",
    };
  },
};
rules["SIA-R69"] = {
  title: "Text minimum contrast",
  severity: "critical",
  check: (d: Document) => this.checkContrast(d, win, false),
};
rules["SIA-R70"] = {
  title: "No obsolete elements",
  severity: "serious",
  check: (d: Document) => {
    const obs = d.querySelectorAll("font, center, marquee");
    return { elements: Array.from(obs), message: "Obsolete tags used" };
  },
};
rules["SIA-R71"] = {
  title: "Paragraphs not justified",
  severity: "serious",
  check: (d: Document) => {
    const ps = d.querySelectorAll("p");
    const warns = Array.from(ps).filter(
      (p) => win.getComputedStyle(p).textAlign === "justify",
```

```
    );
    return { elements: warns, message: "Justified text" };
  },
};
rules["SIA-R72"] = {
  title: "Text in all caps",
  severity: "serious",
  check: (d: Document) => {
    const failures: Element[] = [];

    const elements = Array.from(
      d.querySelectorAll("p, span, li, h1, h2, h3, h4, h5, h6"),
    ).filter((el) => this.isVisible(el, win));

    elements.forEach((el) => {
      const text = el.textContent?.trim();
      if (!text) return;

      // Ignore short strings
      if (text.length < 5) return;

      if (text === text.toUpperCase()) {
        failures.push(el);
      }
    });

    return {
      elements: failures,
      message:
```

```
      failures.length > 0
        ? `${failures.length} element(s) contain all caps text`
        : "No all caps text",
    };
  },
};
rules["SIA-R73"] = {
  title: "Paragraphs line height below minimum",
  severity: "serious",
  check: (d: Document) => {
    const failures: Element[] = [];

    const paragraphs = Array.from(d.querySelectorAll("p")).filter((p) =>
      this.isVisible(p, win),
    );

    paragraphs.forEach((p) => {
      const style = win.getComputedStyle(p);

      const fontSize = parseFloat(style.fontSize);
      const lineHeight = parseFloat(style.lineHeight);

      if (!isNaN(fontSize) && !isNaN(lineHeight)) {
        if (lineHeight < fontSize * 1.5) {
          failures.push(p);
        }
      }
    });
```

```
    return {
      elements: failures,
      message:
        failures.length > 0
          ? `${failures.length} paragraph(s) have insufficient line height`
          : "Line height sufficient",
    };
  },
};
rules["SIA-R74"] = {
  title: "Paragraphs use fixed font size",
  severity: "serious",
  check: (d: Document) => {
    const failures: Element[] = [];

    const elements = Array.from(
      d.querySelectorAll("p, span, li, div"),
    ).filter((el) => this.isVisible(el, win));

    elements.forEach((el) => {
      const style = win.getComputedStyle(el);

      if (style.fontSize.endsWith("px")) {
        failures.push(el);
      }
    });

    return {
      elements: failures,
```

```
      message:
        failures.length > 0
          ? `${failures.length} element(s) use fixed pixel font size`
          : "Font sizes are relative",
    };
  },
};
rules["SIA-R75"] = {
  title: "Font sizes not too small",
  severity: "critical",
  check: (d: Document) => {
    const els = d.querySelectorAll("body *");
    const fails = Array.from(els).filter((el) => {
      const s = parseFloat(win.getComputedStyle(el).fontSize);
      return el.textContent?.trim() && s < 9;
    });
    return {
      passed: fails.length === 0,
      elements: fails,
      message: "Font too small",
    };
  },
};
rules["SIA-R76"] = {
  title: "<th> semantic headers",
  severity: "critical",
  check: (d: Document) => {
    const ths = d.querySelectorAll("th");
    const fails = Array.from(ths).filter(
```

```
      (th) => !th.hasAttribute("scope") && !th.hasAttribute("id"),
    );
    return {
      passed: fails.length === 0,
      elements: fails,
      message: "Missing th mapping",
    };
  },
};
rules["SIA-R77"] = {
  title: "Data cells header cell",
  severity: "critical",
  check: (d: Document) => {
    return { elements: [], message: "Data cell headers" };
  },
};
rules["SIA-R78"] = {
  title: "Content missing after heading",
  severity: "serious",
  check: (d: Document) => {
    const failures: Element[] = [];

    const headings = Array.from(
      d.querySelectorAll("h1, h2, h3, h4, h5, h6"),
    ).filter((h) => this.isVisible(h, win));

    headings.forEach((h) => {
      let next = h.nextElementSibling;
```

```
      if (!next || next.tagName.match(/^H[1-6]$/)) {
        failures.push(h);
      }
    });

    return {
      elements: failures,
      message:
        failures.length > 0
          ? `${failures.length} heading(s) not followed by content`
          : "All headings followed by content",
    };
  },
};
rules["SIA-R79"] = {
  title: "Preformatted code or figure",
  severity: "serious",
  check: (d: Document) => {
    return { elements: [], message: "Pre usage" };
  },
};
rules["SIA-R80"] = {
  title: "Paragraphs relative line height",
  severity: "serious",
  check: (d: Document) => {
    return { elements: [], message: "Absolute line height" };
  },
};
rules["SIA-R81"] = {
```

```
  title: "Links identical names context",
  severity: "serious",
  check: (d: Document) => {
    return { elements: [], message: "Link names" };
  },
};
rules["SIA-R82"] = {
  title: "Error message form description",
  severity: "serious",
  check: (d: Document) => {
    return { elements: [], message: "Error mapping" };
  },
};
rules["SIA-R83"] = {
  title: "Text nodes not clipped resize",
  severity: "serious",
  check: (d: Document) => {
    return { elements: [], message: "Clipping check" };
  },
};
rules["SIA-R84"] = {
  title: "Scrollable keyboard accessible",
  severity: "critical",
  check: (d: Document) => {
    const scrolls = d.querySelectorAll('[style*="overflow: auto"]');
    const fails = Array.from(scrolls).filter(
      (el) => !el.hasAttribute("tabindex"),
    );
    return {
```

```
      passed: fails.length === 0,
      elements: fails,
      message: "Scrollable not keyboard focusable",
    };
  },
};
rules["SIA-R85"] = {
  title: "Paragraphs not all italics",
  severity: "serious",
  check: (d: Document) => {
    return { elements: [], message: "Italic block" };
  },
};
rules["SIA-R86"] = {
  title: "Elements decorative not exposed",
  severity: "critical",
  check: (d: Document) => {
    return { elements: [], message: "Aria-hidden check" };
  },
};
rules["SIA-R87"] = {
  title: "First focusable skip link",
  severity: "critical",
  check: (d: Document) => {
    return { elements: [], message: "Skip link first" };
  },
};
rules["SIA-R88"] = {
  title: "Link text minimum contrast",
```

```
  severity: "critical",
  check: (d: Document) => {
    const links = d.querySelectorAll("a");
    const fails = Array.from(links).filter((l) => {
      const s = win.getComputedStyle(l);
      return this.getContrastRatio(s.color, s.backgroundColor) < 4.5;
    });
    return { elements: fails, message: "Link contrast" };
  },
};
rules["SIA-R89"] = {
  title: "Link text enhanced contrast",
  severity: "serious",
  check: (d: Document) => {
    return { elements: [], message: "Link enhanced contrast" };
  },
};
rules["SIA-R90"] = {
  title: "Presentational children no focusable",
  severity: "critical",
  check: (d: Document) => {
    const pres = d.querySelectorAll('[role="presentation"]');
    const fails = Array.from(pres).filter((p) =>
      p.querySelector("a, button, input"),
    );
    return {
      passed: fails.length === 0,
      elements: fails,
      message: "Interactive inside presentation",
```

```
    };
  },
};
rules["SIA-R91"] = {
  title: "!important letter spacing enough",
  severity: "serious",
  check: (d: Document) => {
    return { elements: [], message: "Letter spacing important" };
  },
};
rules["SIA-R92"] = {
  title: "!important word spacing enough",
  severity: "serious",
  check: (d: Document) => {
    return { elements: [], message: "Word spacing important" };
  },
};
rules["SIA-R93"] = {
  title: "!important line height enough",
  severity: "serious",
  check: (d: Document) => {
    return { elements: [], message: "Line height important" };
  },
};
rules["SIA-R94"] = {
  title: "menuitem has name",
  severity: "critical",
  check: (d: Document) => {
    const ms = d.querySelectorAll('[role="menuitem"]');
```

```
    const fails = Array.from(ms).filter(
      (m) => !this.getAccessibleName(m, doc),
    );
    return {
      passed: fails.length === 0,
      elements: fails,
      message: "Menuitem missing name",
    };
  },
};
rules["SIA-R95"] = {
  title: "<iframe> interactive negative tabindex",
  severity: "critical",
  check: (d: Document) => {
    return { elements: [], message: "Iframe tabindex" };
  },
};
rules["SIA-R96"] = {
  title: "Refreshes meta no delay exception",
  severity: "critical",
  check: (d: Document) => {
    return { elements: [], message: "Meta refresh" };
  },
};
rules["SIA-R97"] = {
  title: "Collapsible blocks content",
  severity: "serious",
  check: (d: Document) => {
    return { elements: [], message: "Collapsible blocks" };
```

```
  },
};
rules["SIA-R98"] = {
  title: "Heading at start main content",
  severity: "serious",
  check: (d: Document) => {
    return { elements: [], message: "Start heading" };
  },
};
rules["SIA-R99"] = {
  title: "Main content inside landmark",
  severity: "critical",
  check: (d: Document) => {
    const main = d.querySelector('main, [role="main"]');
    return {
      passed: !!main,
      elements: !main ? [d.body] : [],
      message: "Missing main landmark",
    };
  },
};
rules["SIA-R100"] = {
  title: "Instrument to main content",
  severity: "critical",
  check: (d: Document) => {
    return { elements: [], message: "Main instrument" };
  },
};
rules["SIA-R101"] = {
```

```
  title: "No repeated content before main",
  severity: "serious",
  check: (d: Document) => {
    return { elements: [], message: "Repeated content" };
  },
};
rules["SIA-R102"] = {
  title: "No repeated content skip link first",
  severity: "critical",
  check: (d: Document) => {
    return { elements: [], message: "Skip link first repeated" };
  },
};
rules["SIA-R103"] = {
  title: "Text in widget minimum contrast",
  severity: "critical",
  check: (d: Document) => {
    return { elements: [], message: "Widget contrast" };
  },
};
rules["SIA-R104"] = {
  title: "Text in widget enhanced contrast",
  severity: "serious",
  check: (d: Document) => {
    return { elements: [], message: "Widget enhanced contrast" };
  },
};
rules["SIA-R109"] = {
  title: "HTML page language matches content",
```

```
  severity: "serious",
  check: (d: Document) => {
    return { elements: [], message: "Lang content match" };
  },
};
rules["SIA-R110"] = {
  title: "Role at least one valid",
  severity: "critical",
  check: (d: Document) => {
    const roles = d.querySelectorAll("[role]");
    const valid = [
      "alert",
      "button",
      "checkbox",
      "dialog",
      "form",
      "grid",
      "heading",
      "img",
      "link",
      "list",
      "main",
      "navigation",
      "region",
      "search",
      "table",
      "textbox",
      "menu",
      "menuitem",
```

```
      "tab",
      "tablist",
      "tabpanel",
    ];
    const fails = Array.from(roles).filter((el) =>
      el
        .getAttribute("role")
        ?.split(" ")
        .every((r) => !valid.includes(r)),
    );
    return {
      passed: fails.length === 0,
      elements: fails,
      message: "Invalid roles set",
    };
  },
};
rules["SIA-R111"] = {
  title: "Target Size enhanced",
  severity: "serious",
  check: (d: Document) => {
    const interactives = Array.from(
      d.querySelectorAll('a, button, [role="button"]'),
    ).filter(
      (el) =>
        this.isVisible(el, win) &&
        !this.isInlineTextLink(el as HTMLElement),
    );
```

```
    const failures: Element[] = [];

    interactives.forEach((el) => {
      const rect = el.getBoundingClientRect();

      if (rect.width < 44 || rect.height < 44) {
        failures.push(el);
      }
    });

    return {
      elements: failures,
      message:
        failures.length > 0
          ? `${failures.length} interactive element(s) below 44x44`
          : "All targets meet enhanced size",
    };
  },
};
rules["SIA-R113"] = {
  title: "Target Size minimum",
  severity: "critical",
  check: (d: Document) => {
    const interactives = Array.from(
      d.querySelectorAll(
        'a, button, [role="button"], input[type="button"], input[type="submit"]',
      ),
    ).filter((el) => {
      if (!(el instanceof HTMLElement)) return false;
```

```
  // Must be visible
  if (!this.isVisible(el, win)) return false;

  // Exclude inline text links
  if (this.isInlineTextLink(el)) return false;

  // Exclude disabled elements
  if ((el as HTMLButtonElement).disabled) return false;

  const style = win.getComputedStyle(el);

  // Exclude pointer-events none
  if (style.pointerEvents === "none") return false;

  return true;
});

const failures: Element[] = [];

interactives.forEach((el) => {
  const rect = el.getBoundingClientRect();

  // Must be BOTH smaller than 24 (not OR)
  if (rect.width < 24 && rect.height < 24) {
    failures.push(el);
  }
});
```

```
    return {
      elements: failures,
      message:
        failures.length > 0
          ? `${failures.length} interactive element(s) below 24x24`
          : "All targets meet minimum size",
    };
  },
};
rules["SIA-R114"] = {
  title: "HTML page title descriptive",
  severity: "serious",
  check: (d: Document) => {
    const t = d.title;
    const descriptive = t && t.length > 10;
    return {
      passed: descriptive,
      elements: !descriptive ? [d.documentElement] : [],
      message: "Title not descriptive",
    };
  },
};
rules["SIA-R115"] = {
  title: "Heading descriptive",
  severity: "serious",
  check: (d: Document) => {
    const hs = d.querySelectorAll("h1, h2, h3, h4, h5, h6");
    const warns = Array.from(hs).filter(
      (h) => (h.textContent?.length || 0) < 3,
```

```
    );
    return { elements: warns, message: "Headings not descriptive" };
  },
};
rules["SIA-R116"] = {
  title: "<summary> has name",
  severity: "critical",
  check: (d: Document) => {
    const ss = d.querySelectorAll("summary");
    const fails = Array.from(ss).filter(
      (s) => !this.getAccessibleName(s, doc),
    );
    return {
      passed: fails.length === 0,
      elements: fails,
      message: "Summary missing name",
    };
  },
};
rules["SIA-R117"] = {
  title: "Image name descriptive",
  severity: "serious",
  check: (d: Document) => {
    const imgs = d.querySelectorAll("img[alt]");
    const warns = Array.from(imgs).filter(
      (i) => (i.getAttribute("alt")?.length || 0) < 3,
    );
    return { elements: warns, message: "Alt text too short" };
  },
```

```
    };

    /*for (let i = 1; i <= 117; i++) {
      const id = `SIA-R${i}`;
      if (!rules[id]) {
        rules[id] = {
          title: `Siteimprove Rule ${id}`,
          severity: 'warning',
          check: (d: Document) => ({ elements: [], message: 'Verified manually or by secondary engine.' })
        };
      }
    }*/

    return rules;
  }
private resolveCategory(ruleId: string): IssueCategory {
  const contentRules = [
    "SIA-R11", // link name
    "SIA-R12",
    "SIA-R57",
    "SIA-R72",
    "SIA-R78"
  ];

  const designRules = [
    "SIA-R60", // contrast
    "SIA-R61",
    "SIA-R111", // target size
    "SIA-R113",
```

```
    "SIA-R73", // line height
    "SIA-R74"
  ];

  const developmentRules = [
    "SIA-R18",
    "SIA-R19",
    "SIA-R20",
    "SIA-R21",
    "SIA-R99"
  ];

  if (contentRules.includes(ruleId)) return "Content";
  if (designRules.includes(ruleId)) return "Design";
  if (developmentRules.includes(ruleId)) return "Development";

  return "General";
}
  async runAllChecks(
    doc: Document,
    win: Window,
  ): Promise<AccessibilityIssue[]> {
    const rules = this.initializeRules(win);
    const results: AccessibilityIssue[] = [];

    for (const ruleId in rules) {
      const rule = rules[ruleId];
      if (!rule) continue;
```

```
try {
  const result = rule.check(doc);
  const mapping = SIAEngine.wcagMapping[ruleId] || {
    sc: ["WCAG"],
    level: "S",
  };
  if (result.elements.length > 0) {
    results.push({
      id: ruleId,
      engine: "alfa",
      impact: rule.severity as Impact,
      description: result.message,
      help: rule.title,
      helpUrl: `https://siteimprove.com/rules/${ruleId.toLowerCase()}`,
      tags: ["Siteimprove", ...mapping.sc],
      wcag: mapping.sc[0],
      issueType: "SIA Rule Violation",
      category: this.resolveCategory(ruleId),
      status: "Confirmed",
      nodes: result.elements.map((el: any) => ({
        html: el.outerHTML?.slice(0, 500) || "DOM Element",
        target: [el.tagName?.toLowerCase() || "element"],
        failureSummary: result.message,
      })),
      confidenceScore: 1.0,
      conformance: mapping.level,
    });
  }
} catch (err) {
```

```
      console.warn(`SIA Rule ${ruleId} execution failed`, err);
    }
  }

  return results;
 }
}

export class ScannerService {
 static detectAemContext(html: string): boolean {
  const indicators = [
    "/etc.clientlibs/",
    "data-sling-resource-type",
    "/content/dam/",
    "cq-panel",
    "foundation-content",
  ];
  return indicators.some((indicator) => html.includes(indicator));
 }

 static async scanRawHtml(
  html: string,
  title: string,
  path: string,
  batchId: string,
  mode: ScanMode,
  onProgress?: (msg: string) => void,
  baseUrl?: string,
  abortSignal?: AbortSignal,
```

```
): Promise<PageScanResult> {
  const isAem = this.detectAemContext(html);
  const iframe = document.createElement("iframe");
  iframe.style.position = "fixed";
  iframe.style.top = "-10000px";
  iframe.style.left = "-10000px";
  iframe.style.width = "1280px";
  iframe.style.height = "1024px";
  document.body.appendChild(iframe);

  return new Promise((resolve, reject) => {
    const onAbort = () => {
      clearTimeout(timeout);
      if (iframe.parentNode) document.body.removeChild(iframe);
      reject(new Error("Audit aborted by user."));
    };
    if (abortSignal?.aborted) return onAbort();
    abortSignal?.addEventListener("abort", onAbort);
    const timeout = setTimeout(() => {
      if (iframe.parentNode) document.body.removeChild(iframe);
      reject(new Error("Accessibility Audit Timed Out."));
    }, 60000);
    iframe.onload = async () => {
      try {
        if (abortSignal?.aborted) return;
        const doc = iframe.contentDocument || iframe.contentWindow?.document;
        const win = iframe.contentWindow;
        if (!doc || !win) throw new Error("Sandbox access failed.");
        // Wait for load
```

```
await new Promise((resolve) => {
  if (doc.readyState === "complete") resolve(null);
  else win.addEventListener("load", () => resolve(null));
});

// Wait for fonts
if (doc.fonts?.ready) {
  await doc.fonts.ready;
}

// Wait for layout stabilization
await new Promise((resolve) =>
  win.requestAnimationFrame(() => win.requestAnimationFrame(resolve)),
);

// Optional micro-delay
await new Promise((r) => setTimeout(r, 50));
// STAGE 1: AXE CORE
onProgress?.("Stage 1: Running Axe Core ...");
await new Promise((r) => setTimeout(r, 500));
if (abortSignal?.aborted) return;

const axeResults = await (win as any).axe.run(doc, {
  runOnly: [
    "wcag2a",
    "wcag2aa",
    "wcag2aaa",
    "wcag21a",
    "wcag21aa",
```

```
    "wcag22a",
    "wcag22aa",
    "best-practice",
  ],
});

const mapAxeIssue = (
  v: any,
  status: IssueStatus,
): AccessibilityIssue => {
  const tags = v.tags as string[];
  let level: ConformanceLevel = "S";
  if (tags.some((t) => t.endsWith("a") && !t.includes("aa")))
    level = "A";
  else if (tags.some((t) => t.endsWith("aa"))) level = "AA";
  else if (tags.some((t) => t.endsWith("aaa"))) level = "AAA";

  return {
    id: v.id,
    engine: "axe",
    impact: (v.impact === "minor" ? "info" : v.impact) as Impact,
    description: v.description,
    help: v.help,
    helpUrl: v.helpUrl,
    tags: [...v.tags, isAem ? "aem-source" : "web-source"],
    wcag:
      v.tags
        .find((t: string) => t.startsWith("wcag"))
        ?.toUpperCase() || "WCAG",
```

```
    issueType: "Axe Core Violation",
    category: v.tags.includes("cat.color")
      ? "Design"
      : v.tags.includes("cat.aria")
        ? "Development"
        : "Content",
    conformance: level,
    status: (axeResults.violations.includes(v)
      ? "Confirmed"
      : "Potential") as IssueStatus,
    nodes: v.nodes.map((n: any) => ({
      html: n.html,
      target: n.target,
      failureSummary: n.failureSummary,
    })),
    confidenceScore: 0.85,
  };
};

const axeIssues = [
  ...axeResults.violations.map((v: any) =>
    mapAxeIssue(v, "Confirmed"),
  ),
  ...axeResults.incomplete.map((v: any) =>
    mapAxeIssue(v, "Potential"),
  ),
];

// STAGE 2: COMPREHENSIVE SIA ENGINE
```

```
  if (abortSignal?.aborted) return;
  onProgress?.(
    "Stage 2: Analyzing with SIA Alfa Custom Accessibilty Engine...",
  );
  const siaEngine = new SIAEngine();
  const siaIssues = await siaEngine.runAllChecks(doc, win);

  const result: PageScanResult = {
    scanId: crypto.randomUUID
      ? crypto.randomUUID()
      : Math.random().toString(36).substring(2, 15),
    batchId,
    mode,
    path:
      path || (isAem ? "AEM Page Content" : "HTML File") || "Source",
    title: doc.title || title || "Audit Result",
    url: "local://audit",
    issues: [...axeIssues, ...siaIssues],
    timestamp: Date.now(),
    htmlSnapshot: html,
  };

  clearTimeout(timeout);
  abortSignal?.removeEventListener("abort", onAbort);
  document.body.removeChild(iframe);
  resolve(result);
} catch (err) {
  clearTimeout(timeout);
  abortSignal?.removeEventListener("abort", onAbort);
```

```
      if (iframe.parentNode) document.body.removeChild(iframe);
      reject(err);
    }
  };

  const axeScriptUrl =
    "https://cdn.jsdelivr.net/npm/axe-core@4.11.1/axe.min.js";

  let processedHtml = html.includes("<html")
    ? html
    : `<!DOCTYPE html><html
lang="en"><head><title>${title}</title></head><body>${html}</body></html>`;
  if (baseUrl) {
    processedHtml = processedHtml.replace(
      "<head>",
      `<head><base href="${baseUrl}">`,
    );
  }
 /* iframe.srcdoc = `
    <!DOCTYPE html>
    <html>
      <head><script src="${axeScriptUrl}"></script></head>
      <body>${processedHtml}</body>
    </html>
  `;*/
  let finalHtml = processedHtml;

  // Inject axe into existing <head>
  if (finalHtml.includes("<head")) {
```

```
      finalHtml = finalHtml.replace(
        "<head>",
        `<head><script src="${axeScriptUrl}"></script>`
      );
    } else {
      // fallback if no head exists
      finalHtml = `
        <!DOCTYPE html>
        <html>
          <head><script src="${axeScriptUrl}"></script></head>
          <body>${processedHtml}</body>
        </html>
      `;
    }

    iframe.srcdoc = finalHtml;
  });
}

static async scanPage(
  path: string,
  batchId: string,
  mode: ScanMode,
  onProgress?: (msg: string) => void,
  abortSignal?: AbortSignal,
): Promise<PageScanResult> {
  onProgress?.(`Fetching content for: ${path}`);

  let targetUrl = path.trim();
```

```
if (!targetUrl.startsWith("http")) {
  targetUrl = `https://${targetUrl}`;
}

try {
  const response = await fetch(
    `https://api.allorigins.win/raw?url=${encodeURIComponent(targetUrl)}`,
    { signal: abortSignal },
  );

  if (!response.ok) {
    throw new Error(
      `Proxy error: ${response.status} ${response.statusText}`,
    );
  }

  const html = await response.text();

  if (!html || html.trim().length < 50) {
    throw new Error("Received empty or blocked HTML content.");
  }

  let baseUrl: string | undefined;
  try {
    const urlObj = new URL(targetUrl);
    baseUrl =
      urlObj.origin +
      urlObj.pathname.substring(0, urlObj.pathname.lastIndexOf("/") + 1);
  } catch {}
```

```
      return this.scanRawHtml(
        html,
        "AEM Page Audit",
        targetUrl,
        batchId,
        mode,
        onProgress,
        baseUrl,
        abortSignal,
      );
    } catch (error) {
      if (abortSignal?.aborted) {
        throw new Error("Fetch aborted.");
      }

      throw new Error(
        `Accessibility engine could not connect to: ${targetUrl}. Technical reason: ${
          (error as Error).message
        }`,
      );
    }
  }
}
```